



OpenWorld on ARC-AGI-3: Synthesizing Verified Code World Models for Interactive Reasoning

James Schwoebel^{1,*} • Ingrida Semenec, PhD¹ • Jenia Rousseva, PhD¹ •
Marcos Ortiz, PhD¹ • Collin Overbay¹ • Manish Bhatt^{4,6} • Rome Thorstenson³ •
Martin G. Frasnch, MD, PhD^{2,5}

¹ Quome, Inc. ² University of Washington ³ Rafter Labs, Inc. ⁴ OWASP ⁵ Health Stream Analytics, LLC
⁶ IEEE Computer Society * Corresponding author

ABSTRACT

ARC-AGI-3 is the ARC Prize Foundation’s interactive-reasoning benchmark: an agent is dropped into an unfamiliar grid game, must discover the rules *by acting*, and reach goals over many steps—testing exploration, *modeling*, goal-setting, and planning. Frontier agents score near zero. We argue this is precisely the regime where a *world computer* should win, and give the OPENWORLD recipe: **explore** a game to gather verified (s, a, s') transitions, **synthesize** the game’s dynamics as *verified code* (an LLM writes a `predict(frame, action)` function, accepted only after it exact-matches held-out transitions), and **plan** through the verified model to reach goals. The wager is the same one that powers symbolic code world models elsewhere: a *correct* code model is exact and admits search, where a learned next-frame model compounds error. A precondition holds empirically—on the public game `1s20`, observed dynamics are *deterministic* (every repeated state-action pair yields an identical next frame across hundreds of transitions), so the dynamics are writable as exact code; states are dense ($\sim 1,500$ of 4,096 cells non-background) but per-step changes are sparse (~ 50 cells), which a code model handles natively while a token-level learner cannot. [Results in progress: this manuscript reports the recipe, the determinism precondition, and the agent; quantitative level-completion vs. random and learned-model baselines across the 25 public games are being measured.]

Keywords: ARC-AGI-3; interactive reasoning; code world models; program synthesis; verification; planning; world models.

1 Introduction

The interactive frontier. ARC-AGI-1 and -2 measure abstraction from static input-output grids; ARC-AGI-3 moves to *interactive* environments where the rules and goals are revealed only through action. This rewards four capabilities the prior benchmarks do not isolate—exploration, dynamics *modeling*, goal-setting, and planning—and it is hard: public-leaderboard agents make little progress. Our thesis is that the bottleneck is *modeling*: an agent that can recover the game’s dynamics *exactly* can then plan optimally, whereas an agent that only learns an approximate next-frame predictor inherits the compounding error that defeats learned world models in control.

This paper. We port OPENWORLD’s verified-code-world-model recipe to ARC-AGI-3. The contribution is (i) the *recipe*—explore, synthesize-and-verify code dynamics, plan—and the argument for why it fits an interactive grid benchmark; (ii) the empirical *precondition* that the games

are deterministic and therefore code-expressible; and (iii) [in progress] a measured comparison of level completion against random and learned-model baselines on the public games.

2 The ARC-AGI-3 setting

Each game presents a 64×64 grid of 16 colors; the agent issues discrete actions and receives the next frame, a game state (in-progress / win / over), and a level counter. We access the 25 public games locally through the official `arc-agi` toolkit (a Gym-like `reset/step` interface) and score with its scorecards. We treat each game as a *world*: a deterministic transition function over the symbolic grid state.

3 The recipe: explore, synthesize-and-verify, plan

Explore. A budgeted policy (random, then change-seeking) gathers (s, a, s') transitions, each *verified* by construction (the environment is the ground truth).

Synthesize verified code. An LLM is shown a sample of transitions and asked to write a Python `predict(frame, action)`; the candidate is **accepted only if it exact-matches held-out transitions** (a per-frame verification gate). Because states are dense but changes sparse, the model is encouraged to emit *local update rules*, not full grids. Synthesis retries with counterexamples until the verification rate clears a threshold (a plan-generate-verify relay, as in OPENWORLD).

Plan. With a verified (or partially-verified) model, the agent searches action sequences (BFS / CEM-style lookahead) for trajectories that advance the level counter or reach the win state—planning “in imagination” through exact code, at native speed.

4 Preliminary findings

On 1s20: (1) **integration** works end-to-end (local play, no API key); (2) **determinism** is exact—1.00 of repeated state-action pairs reproduce the next frame; (3) **baselines** (random and a change-seeking heuristic) complete 0 of the game’s levels in a 400-step budget, confirming headroom; (4) states are dense ($\sim 1,500$ non-background cells) but per-step deltas are small (~ 50 cells), which motivates the code-synthesis representation over token-level frame prediction. Full results across games are in progress.

5 Reproducibility

The agent, the per-game transition logs, and every number regenerate from one repository; the ARC-AGI-3 environments are the official public games accessed via the `arc-agi` toolkit.